

PrayogX Content Architecture

One canonical library, four clients. What holds, what breaks at scale, and the one defect that fires the next time you add a simulation.

COMMIT	FEED VERSION	SIMULATIONS	PUBLISHED/SIM	SUITES GREEN
48b25d6	33505a9321bd	17	143 KB	13 / 419

The architecture is sound and does not need redesigning. One canonical source, one derive step, one feed, two clients reading it and a third that can join without touching any of it. Every constraint you set is actually enforced by a tool, not by discipline — I tried to break each one and the tooling stopped me.

One real defect. Adding simulation #18 deletes every simulation your students have saved for offline use on the website and PWA. Not a risk — I reproduced it. Android is unaffected. Two lines in `sw.js`.

A CONTENT ARCHITECTURE

One source, one derive step, one lock

01	data/manifest.json Authoring source of truth. 167 KB, 9.9 KB per simulation — verification prose, interactivity notes, concepts. Never shipped to a client as the catalogue.
02	tools/sync_manifest.py Sorts by id, refreshes the counts block, mirrors to <code>manifest.js</code> . It never assigns, rennumbers or rewrites an id — I checked; the only thing it does to ids is sort on them.
03	tools/build_content.py Derives the client feed and the crawlable pages. The feed version is a hash of the card records, so it moves when and only when a card changes.
04	content/ <code>catalog.json</code> 1.4 KB · <code>index.json</code> 1,197 B/sim · <code>search.json</code> 547 B/sim, lazy · <code>sims/<ID>.json</code> 8.5 KB each, on demand.
05	data/revisions.json The lock: <code>{id: {revision, sha256}}</code> . An edit to a simulation without a revision bump fails <code>check_library.py</code> — so a change nobody would see cannot be published.

The split is the thing that makes this scale. A manifest entry averages 9.9 KB; a card record averages 1,197 B. Shipping the manifest would mean 9.9 MB before the first card paints at 1,000 simulations. Shipping cards means 1.2 MB. That decision is already made and already correct.

FROM SOURCE `check_library.py` enforces, and fails the build on: missing required fields, duplicate ids, ids that do not match `ADV-<YEAR>-P<PAPER>-<SUBJ>-Q<NN>`, broken paths, missing companion files, `meta.json`

disagreeing with the manifest, stale counts, a simulation published but absent from the feed, a feed record with no manifest entry, a stale detail record, a missing crawlable page, a `sw.js` version stamp that does not match the feed, an unbumped revision, and orphan folders. That is a genuinely complete set.

Cache invalidation. Every simulation URL carries `?v=<revision>`, so a revised simulation is a different URL to every cache in the chain. The Android app stores offline copies in IndexedDB keyed by id with the revision in the record, and re-fetches only when they differ. Both correct. The service worker is where it goes wrong — see **N1**.

B WEB

Pages deployment, routing, rendering

Deployment. A GitHub Actions workflow with a *deny* list rather than an allow list, so a new directory is published by default and cannot be forgotten. Before it assembles anything it re-runs sync, build and check, then refuses to deploy if the generated content differs from what was committed. It also fails hard if `papers/` or any PDF reaches the deploy directory. This is the strongest part of the release path.

Routing. Hash routing — `#/` for the catalogue with filters in the hash query, `#/sim/<ID>` for a detail page. Bookmarkable, no server rewrites needed, correct for a static host. Simulation links are relative paths plus `?v=`; the app alone uses absolute URLs, built from the single `origin` line in `config.js`. Nothing is hardcoded twice.

Rendering. **MEASURED** The grid draws 24 cards and holds the rest behind *Show more*. At a synthetic 1,000 simulations the DOM still contained exactly 24 cards and first paint was 702 ms. The boot chain degrades properly: `content/index.json`, then `data/manifest.json`, then an injected `data/manifest.js` for `file://`. The heavy fallback is only fetched if the light one fails.

Search. This is the one web bottleneck, and it is a real one — see **L1**.

C ANDROID

A client, not a copy

No duplicated catalogue. **MEASURED** The bundle is four files — `index.html`, `app.js`, `app.css`, `config.js`. `production_audit.py` asserts there is no `content/` or `data/` directory inside `app/www`, and `appcheck_prod.js` watches every request the shell makes and fails if any simulation or catalogue file comes from the app's own origin. The app ships zero simulations.

Adding a simulation needs no rebuild and no store release. This is proven every test run, not asserted: `propagation.js` adds a real simulation to `data/manifest.json`, runs the real pipeline, and checks that the website, the search index, the revision-tagged link, the detail page, the generated crawlable page, the PWA cache and the Capacitor app all pick it up with no code change — then removes it and confirms the feed version returns to the identical string.

Offline. Catalogue and index kept in `localStorage`; opened simulations in IndexedDB, newest 30 kept. A simulation is a single self-contained HTML file rendered with `srcdoc`, which is why an opened one keeps working with no connection. **MEASURED** I confirmed all 17 simulation files are genuinely self-contained: no `<script src>`, no external stylesheet, no `@import`, no `fetch()`, no `localStorage`. That property is load-bearing for offline, for `srcdoc`, and for iOS later.

API 36. `compileSdk 36 / targetSdk 36 / minSdk 22` on AGP 8.10.0, Gradle 8.11.1, Capacitor 6.2.2, and it builds and runs. Edge-to-edge is handled by handing the real `WindowInsets` to the four CSS custom properties the shell

already read for iOS. `CapacitorHttp` is enabled, so the content host is never asked for CORS headers — that removed a whole class of failure.

D FUTURE IOS

No architectural blockers — only work

The content system needs no change at all for iOS. Everything platform-specific is already either shared or absent:

- `capacitor.config.json` already carries its `ios` block (`contentInset: "always"` , app-bound domains off).
- `app.css` reads `env(safe-area-inset-*)` natively — iOS fills those in itself, so `MainActivity` 's Android-only override is not needed and not in the way.
- `srcdoc` , `IndexedDB` , `localStorage` and `CapacitorHttp` all work in `WKWebView`.
- The app fetches the same feed from the same `origin` line.

What remains is `npx cap add ios` on macOS, signing, artwork and a listing. Two things to know before you submit, neither of which is a redesign:

- **App Store review 2.5.2 / 4.7.** The app fetches HTML and JavaScript at runtime and runs it in a web view. That is permitted — it is web content in a web view, and the app itself ships no executable updates — but it is a conversation reviewers sometimes start. Have the answer ready rather than being surprised by it.
- **WKWebView data persistence.** Safari's seven-day eviction of unused website data does not apply to an app's own `WKWebView` store, but VERIFY this on a device before you promise students that downloaded simulations stay downloaded.

E SCALABILITY

Where the numbers actually go

Measured from the real 17-simulation library, projected linearly. Every per-simulation figure below is a division of real bytes on disk, not an estimate.

COMPONENT	PER SIM	17	100	500	1,000	5,000
catalog.json revisions	24 B	0.4 KB	2 KB	12 KB	24 KB	120 KB
index.json (<i>startup</i>)	1,197 B	20 KB	117 KB	584 KB	1.17 MB	5.8 MB
search.json (<i>lazy</i>)	547 B	9 KB	53 KB	267 KB	534 KB	2.7 MB
content/sims/ (<i>on demand</i>)	8.5 KB	144 KB	846 KB	4.2 MB	8.5 MB	42 MB
simulations/ (<i>incl. meta + question</i>)	107.5 KB	1.83 MB	10.8 MB	54 MB	108 MB	538 MB
s/ crawlable stubs	5.7 KB	97 KB	569 KB	2.8 MB	5.7 MB	28 MB
data/manifest.json + .js	19.7 KB	335 KB	1.97 MB	9.9 MB	19.7 MB	99 MB
Site total	143 KB	2.4 MB	14.3 MB	72 MB	143 MB	716 MB
Published bytes by component. "Site total" is everything GitHub Pages serves.						

WHAT	RESULT		READING
Startup transfer	1.26 MB	Index, not manifest — the split is working	
First paint	702 ms	Fine	
Cards in the DOM	24	Pagination holding	
Filter a chapter	422 ms	Acceptable	
Full-text search	1,428 ms	The wall. Linear scan, every keystroke	
Behaviour at 1,000 simulations, measured in a headless browser against a synthetic catalogue of that size.			

What this means in practice. At **17** and **100** nothing is a problem anywhere. At **500** search becomes noticeably sluggish and the startup index passes half a megabyte. At **1,000** search is the only thing a student would complain about. The hard ceiling is GitHub Pages' **1 GB published-site limit**, which this layout reaches at roughly **7,000 simulations** — and the bandwidth soft limit of 100 GB/month is about 1.1 million simulation opens, so it is not the binding constraint. When Pages does run out, `config.js` is one line pointing at the content host; moving to a CDN is a config change, not a migration. That is the architecture paying off.

Device memory and startup. A simulation averages 91 KB of HTML and renders in an iframe from a string; only one is live at a time. The offline store is capped at 30 on Android and 40 on the web, so worst case is about 3.6 MB of stored HTML. Neither grows with library size. App startup fetches only `catalog.json` when the feed version has not moved — 24 KB at 1,000 — so launch cost is almost flat.

F RELEASE SAFETY

What adding simulation #18 actually does

CAN ADDING #18...	VERDICT		WHY
break an existing simulation?	No	Each is a self-contained file; nothing shares code	
change an existing id?	No	Ids are author-supplied; sync only sorts; duplicates fail the check	
invalidate unrelated content?	Yes	N1 — the service worker drops saved simulations	
create stale Android content?	No	Catalogue fetched no-store ; sims keyed by revision	
create stale web content?	No	Shell and feed caches are version-keyed; the workflow refuses stale builds	
Each row checked against the running system, not the intention.			

Four of five are genuinely safe, and safe because a tool enforces them rather than because the process is careful. The fifth is the defect below.

Four buckets, exact files

● MUST FIX NOW

1 item

N1 Adding a simulation deletes every simulation saved for offline use on the web

REPRODUCED

`sw.js` names all three caches after the feed version — `var SIMS = "prayogx-sims-" + VERSION;` — and the `activate` handler deletes every `prayogx-*` cache whose name does not contain the current version. Adding a simulation changes the card hash, which changes the feed version, which renames the simulations cache, which deletes the old one.

I ran it: opened the site, opened a simulation, confirmed `prayogx-sims-33505a9321bd` held 1 entry. Added a simulation through the real pipeline. After the new worker activated, the sims caches were `[]`. Every simulation a student had saved for the bus home, gone, the next time you publish.

Versioning the shell and feed caches is right — those must be replaced. The sims cache must not be, because its entries are *already* immutable: each one is keyed by a URL containing `?v=<revision>`, so a revised simulation is a different entry and a stale one is unreachable. A stable name plus the existing cap is all it needs.

Android is unaffected — it stores offline copies in IndexedDB by id with the revision inside the record, independent of the feed version. Only the website and PWA lose data.

FILES

```
sw.js — name the sims cache stably; exempt it from the activate sweep (2 lines)
tools/production_audit.py — assert the sims cache name carries no version
tests/stalecache.js — a case that opens a simulation, adds another, and requires the first to survive
```

● SHOULD FIX BEFORE PLAY STORE

3 items

S1 versionCode is still 1

FROM SOURCE

Play rejects any upload whose `versionCode` does not exceed the last one. It is 1 today with `versionName "1.0"`, which is correct for a first upload and wrong for every one after. Worth a note in the release checklist rather than a code change now.

FILES

```
app/android/app/build.gradle — versionCode, versionName
```

s2 Status bar icons may be invisible in light theme

NOT TESTED

`capacitor.config.json` pins `StatusBar.style: "DARK"`, which means light icons. That was right when the bar had its own `#0d1524` background — but `StatusBar.backgroundColor` is a no-op on Android 15+, and under edge-to-edge the icons now sit on the header's own colour, which is white in light theme. Check it on the emulator in light mode before you ship. If it is wrong, fix it in JavaScript where the theme is already known, not in the config: that `style` value is shared with iOS.

FILES

`app/www/app.js` — call `StatusBar.setStyle` beside the existing theme toggle

s3 No icon or splash artwork

FROM SOURCE

Still the generated Capacitor placeholders. `@capacitor/assets` needs a 1024×1024 icon and a 2732×2732 splash and generates every density from them.

FILES

`app/assets/icon.png`, `app/assets/splash.png` (new)

● CAN WAIT

5 items

L1 Search is the first real scaling wall — around 300 simulations

MEASURED

1,428 ms at 1,000, because every query is a linear scan over every blob. It is invisible at 17 and fine to roughly 300. The fix when you get there is a prebuilt inverted index in the feed — token to id list, built once at publish time instead of scanned on every keystroke — which also shrinks what the client downloads. Do not do it yet; it is real work and it buys nothing at your current size.

FILES

`tools/build_content.py` — emit `content/search-index.json`
`site/site.js`, `app/www/app.js` — look up instead of scanning

L2 Every addition makes every client re-download the whole index

FROM SOURCE

One simulation added moves the feed version, and both clients then fetch `index.json` in full — 1.17 MB at 1,000. If you add simulations weekly, that is 1.17 MB per student per week for one new card. Harmless now; worth shaping around 500. The shape of the fix is sharding the index by year or subject, or shipping a delta keyed on the previous version.

FILES

`tools/build_content.py`, `site/site.js`, `app/www/app.js`

L3 28% of the published site is files no client ever requests

MEASURED

Each simulation folder publishes `meta.json` (9.1 KB) and `question.md` (7.3 KB) beside the simulation, and `data/` publishes both `manifest.json` and `manifest.js` (19.7 KB per simulation between them). That is 36 KB per simulation of authoring material against 91 KB of actual simulation. Nothing private leaks — the exam PDFs are excluded and the workflow fails if one appears — it is just weight: 16 MB of meta and question files at 1,000, and 20 MB of manifests. Note `manifest.json` is a live fallback the site fetches when the feed fails, so it cannot simply be dropped; the companion files can.

FILES

```
.github/workflows/static.yml – exclude the companion files from the deploy
```

L4 Detail records are not revision-keyed, unlike simulations

FROM SOURCE

`content/sims/<ID>.json` is fetched at a bare URL while the simulation HTML carries `?v=<revision>`. Stale-while-revalidate corrects it on the next view, so the visible effect is a briefly out-of-date description — but it is the one place the revision discipline is not applied end to end, and consistency here is cheap.

FILES

```
tools/build_content.py, site/site.js, app/www/app.js
```

L5 The offline cache evicts oldest-opened, not least-recently-used

FROM SOURCE

`trim()` in `sw.js` deletes from the front of `cache.keys()`, which is insertion order — so it is FIFO, while the comment above it says LRU. A student who reopens Q1 every day still loses it once 40 newer simulations arrive. Small, and the comment is wrong either way; correct the comment now and the behaviour whenever you next touch the file.

FILES

```
sw.js – trim()
```

K1-K7 The parts carrying the whole design

K1 · `data/manifest.json` as the only source, with everything else derived. This is the decision the rest depends on.

K2 · The revision lock. `data/revisions.json` plus `check_library.py` 's refusal of an unbumped edit is what makes cache invalidation reliable rather than hopeful.

K3 · Self-contained single-file simulations. No CDN, no fonts, no fetch, no storage, no build step. This is what makes `srcdoc` , offline rendering and the iOS story all work at once. Verified across all 17.

K4 · `config.js` as the single line pointing every client at the content host. It is also your escape hatch when Pages runs out.

K5 · The app shipping no catalogue and no simulation. Asserted by the audit tool and watched request-by-request by the shell suite.

K6 · The deny-list deploy plus the stale-content refusal. A new directory is published by default; a feed that does not match its source is never published at all.

K7 · The current toolchain. Capacitor 6.2.2, AGP 8.10.0, Gradle 8.11.1, `compileSdk/targetSdk 36`, `minSdk 22`. It builds, it runs on API 36, and it meets the Play floor. Do not move any of it until Play forces you to.

Audited at commit `48b25d6` , feed version `33505a9321bd` , 17 simulations. Claims marked **MEASURED** or **REPRODUCED** were established by running something; **FROM SOURCE** by reading the file named; **NOT TESTED** could not be checked from here and needs a device. Nothing in the repository was modified — the working tree is clean at `48b25d6` , and the one diagnostic that added a simulation removed it again and restored the feed version to the identical string.